

# GAN-Based Anomaly Detection & Graph Anomaly Detection

Lecture Note — Module 2, Block 4

AI4STEM

---

## Contents

|           |                                                                           |          |
|-----------|---------------------------------------------------------------------------|----------|
| <b>I</b>  | <b>GAN-Based Anomaly Detection</b>                                        | <b>1</b> |
| 1         | Generative Adversarial Networks: Foundations                              | 1        |
| 2         | AnoGAN: Anomaly Detection via Latent Space Search                         | 3        |
| 3         | f-AnoGAN: Fast Inference via Learned Encoding                             | 4        |
| 4         | Wasserstein GAN with Gradient Penalty (WGAN-GP)                           | 5        |
| <b>II</b> | <b>Graph Anomaly Detection</b>                                            | <b>6</b> |
| 5         | Graphs: Notation and Terminology                                          | 6        |
| 6         | Types of Graph Anomalies                                                  | 7        |
| 6.1       | Node-Level Anomalies . . . . .                                            | 7        |
| 6.2       | Edge-Level Anomalies . . . . .                                            | 8        |
| 6.3       | Subgraph-Level Anomalies . . . . .                                        | 9        |
| 6.4       | Whole-Graph-Level Anomalies . . . . .                                     | 10       |
| 7         | Feature-Based Graph Anomaly Detection                                     | 11       |
| 7.1       | Structural Node Features . . . . .                                        | 11       |
| 8         | Spectral Methods for Graph Anomaly Detection                              | 12       |
| 8.1       | The Graph Laplacian . . . . .                                             | 13       |
| 8.2       | Spectral Anomaly Detection . . . . .                                      | 13       |
| 9         | Deep Graph Anomaly Detection: DOMINANT                                    | 14       |
| 9.1       | Graph Convolutional Networks (GCN) . . . . .                              | 14       |
| 9.1.1     | From spectral filters to a localised polynomial . . . . .                 | 14       |
| 9.1.2     | The renormalisation trick: deriving the GCN layer . . . . .               | 14       |
| 9.1.3     | Node-wise rewriting and message passing . . . . .                         | 15       |
| 9.1.4     | Receptive field, computational cost, and the role of self-loops . . . . . | 16       |
| 9.1.5     | Failure modes and limitations . . . . .                                   | 16       |
| 9.2       | DOMINANT Architecture . . . . .                                           | 17       |
| 10        | Other Deep Graph Anomaly Detection Methods                                | 18       |
| 10.1      | Contrastive Methods . . . . .                                             | 18       |
| 10.2      | Dual Autoencoder Methods . . . . .                                        | 18       |

## 11 Method Selection Guide

19

Table 1: \*

|          | Symbol                                     | Meaning                                                               |
|----------|--------------------------------------------|-----------------------------------------------------------------------|
| Notation | <i>GAN-related</i>                         |                                                                       |
|          | $G$                                        | Generator network                                                     |
|          | $D$                                        | Discriminator network                                                 |
|          | $C$                                        | Critic (Wasserstein setting)                                          |
|          | $E$                                        | Encoder (f-AnoGAN)                                                    |
|          | $p_z$                                      | Latent noise prior (typically $\mathcal{N}(\mathbf{0}, \mathbf{I})$ ) |
|          | $p_{\text{data}}, p_G$                     | Data distribution and generator distribution                          |
|          | $f_D(\cdot)$                               | Intermediate features of discriminator                                |
|          | $\kappa, \lambda$                          | Feature-matching and gradient-penalty weights                         |
|          | <i>Graph-related</i>                       |                                                                       |
|          | $\mathcal{G} = (V, E, \mathbf{X})$         | Attributed graph                                                      |
|          | $\mathbf{A} \in \{0, 1\}^{n \times n}$     | Adjacency matrix                                                      |
|          | $\mathbf{D}$                               | Degree matrix                                                         |
|          | $\mathbf{L}_{\text{sym}}$                  | Normalised graph Laplacian                                            |
|          | $\mathbf{X} \in \mathbb{R}^{n \times d}$   | Node feature matrix                                                   |
|          | $\mathbf{Z} \in \mathbb{R}^{n \times d_z}$ | Node embedding matrix                                                 |
|          | $\hat{\mathbf{A}}$                         | Normalised adjacency (with self-loops)                                |
|          | $\alpha$                                   | Attribute vs structure weight (DOMINANT)                              |

## Part I

## GAN-Based Anomaly Detection

## 1 Generative Adversarial Networks: Foundations

## Definition 1.1: Generative Adversarial Network (GAN)

A **Generative Adversarial Network** [1] consists of two neural networks trained simultaneously in a competitive game:

1. A **generator**  $G : \mathbb{R}^{d_z} \rightarrow \mathbb{R}^{d_x}$  that maps a latent noise vector  $\mathbf{z} \sim p_z$  (typically  $p_z = \mathcal{N}(\mathbf{0}, \mathbf{I})$ ) to a synthetic sample  $\tilde{\mathbf{x}} = G(\mathbf{z})$ .
2. A **discriminator**  $D : \mathbb{R}^{d_x} \rightarrow [0, 1]$  that estimates the probability that a given sample is drawn from the real data distribution  $p_{\text{data}}$  rather than from  $G$ .

## Theorem 1.1: GAN Minimax Objective

The generator and discriminator are trained according to the minimax game:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))] \quad (1)$$

**Theorem 1.2: Nash Equilibrium of the GAN Game**

For the non-parametric case (i.e. when  $G$  and  $D$  have unlimited capacity), the minimax game in Eq. (1) has a unique Nash equilibrium characterised by:

1. The generator's distribution matches the data distribution:  $p_G = p_{\text{data}}$ .
2. The optimal discriminator satisfies  $D^*(\mathbf{x}) = \frac{1}{2}$  for all  $\mathbf{x}$ .

At equilibrium, the value of the game is  $V(D^*, G^*) = -\log 4$ , and the Jensen–Shannon divergence between  $p_G$  and  $p_{\text{data}}$  is zero.

*Proof sketch.* For fixed  $G$ , the optimal discriminator is  $D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}$ . Substituting this into the value function yields:

$$V(D_G^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_G),$$

where JSD denotes the Jensen–Shannon divergence. Since  $\text{JSD} \geq 0$  with equality if and only if  $p_G = p_{\text{data}}$ , the global minimum over  $G$  is achieved at  $p_G = p_{\text{data}}$ .  $\square$

**Proposition 1.1: Optimal Discriminator**

For a fixed generator  $G$ , the optimal discriminator  $D_G^*$  is:

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}. \quad (2)$$

*Proof.* For fixed  $G$ , the value function is

$$V(D, G) = \int_{\mathbf{x}} [p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_G(\mathbf{x}) \log(1 - D(\mathbf{x}))] d\mathbf{x}.$$

Pointwise, the integrand  $a \log y + b \log(1 - y)$  (with  $a = p_{\text{data}}(\mathbf{x})$ ,  $b = p_G(\mathbf{x})$ ) is maximised at  $y = a/(a + b)$ . This is verified by setting the derivative to zero:  $a/y - b/(1 - y) = 0$  yields  $y^* = a/(a + b)$ .  $\square$

**Remark 1.1: Mode collapse**

In practice, the GAN game does not always converge to the theoretical Nash equilibrium. A common failure mode is **mode collapse**, where the generator learns to produce only a small subset of the modes of  $p_{\text{data}}$ . For anomaly detection, mode collapse is particularly dangerous: if the generator fails to cover parts of the normal manifold, normal samples from uncovered modes will receive high anomaly scores (false positives). WGAN-GP (Section 4) substantially mitigates this issue.

**Remark 1.2: Why GANs for anomaly detection**

A well-trained GAN implicitly learns the manifold of the training distribution. When trained on normal data only, the generator captures  $p_{\text{normal}}$  and the discriminator learns to recognise samples from this distribution. Anomalies, by definition, lie off this manifold. This insight underpins all GAN-based anomaly detection methods.

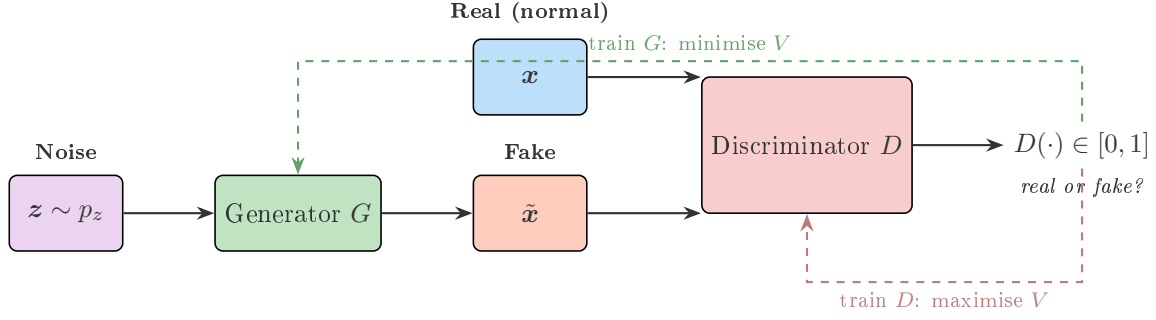


Figure 1: GAN adversarial training loop. The generator  $G$  produces fake samples from noise; the discriminator  $D$  tries to distinguish them from real data. The two networks are trained alternately in a minimax game.

## 2 AnoGAN: Anomaly Detection via Latent Space Search

### Definition 2.1: AnoGAN

**AnoGAN** [2] detects anomalies by measuring how well a query sample  $\mathbf{x}$  can be represented in the learned latent space of a trained generator  $G^*$ . The anomaly score involves solving an optimisation problem at inference time:

$$\mathbf{z}^* = \arg \min_z \underbrace{\|\mathbf{x} - G(\mathbf{z})\|^2}_{\text{reconstruction loss}} + \lambda \underbrace{\|f_D(\mathbf{x}) - f_D(G(\mathbf{z}))\|^2}_{\text{feature-matching loss}}, \quad (3)$$

where  $f_D(\cdot)$  denotes the activations of an intermediate layer of the discriminator  $D$ , and  $\lambda > 0$  balances the two terms. The anomaly score is:

$$s_{\text{AnoGAN}}(\mathbf{x}) = \|\mathbf{x} - G(\mathbf{z}^*)\|^2 + \lambda \|f_D(\mathbf{x}) - f_D(G(\mathbf{z}^*))\|^2. \quad (4)$$

### Remark 2.1: Computational cost of AnoGAN

The optimisation in Eq. (3) must be performed for *every* query sample at inference time, typically requiring several hundred gradient descent steps. This makes AnoGAN prohibitively slow for real-time or large-scale applications. This limitation motivates the development of f-AnoGAN.

The intuition behind AnoGAN is as follows. For a normal sample  $\mathbf{x} \sim p_{\text{normal}}$ , there should exist some  $\mathbf{z}$  in the latent space such that  $G(\mathbf{z}) \approx \mathbf{x}$ , because  $G$  has learned to generate the normal distribution. For an anomalous sample, no such  $\mathbf{z}$  exists, so the reconstruction is poor and the anomaly score is high.

The feature-matching term  $\|f_D(\mathbf{x}) - f_D(G(\mathbf{z}))\|^2$  provides a perceptually richer comparison than pixel-wise distance alone: it measures similarity in the discriminator's learned feature space, capturing higher-level structural patterns.

### 3 f-AnoGAN: Fast Inference via Learned Encoding

#### Definition 3.1: f-AnoGAN

**f-AnoGAN** [3] accelerates AnoGAN by training an additional **encoder network**  $E : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_z}$  that learns to invert the generator, eliminating the need for per-sample optimisation. Training proceeds in two phases:

1. **Phase 1:** Train the GAN  $(G, D)$  on normal data as usual.
2. **Phase 2:** Freeze  $(G^*, D^*)$  and train  $E$  to minimise:

$$\mathcal{L}_E = \|x - G(E(x))\|^2 + \kappa \|f_D(x) - f_D(G(E(x)))\|^2, \quad (5)$$

where  $\kappa > 0$  is a hyperparameter.

#### Definition 3.2: f-AnoGAN Anomaly Score

The **f-AnoGAN anomaly score** for a query  $x$  is:

$$s_{\text{f-AnoGAN}}(x) = \underbrace{\|x - G(E(x))\|^2}_{\text{image distance}} + \kappa \underbrace{\|f_D(x) - f_D(G(E(x)))\|^2}_{\text{feature distance}}. \quad (6)$$

This score requires only a single forward pass through  $E$ ,  $G$ , and  $D$ —no iterative optimisation.

#### Proposition 3.1: f-AnoGAN vs AnoGAN: inference complexity

Let  $K$  denote the number of gradient steps required by AnoGAN’s latent optimisation (Eq. 3). Each step involves a forward and backward pass through  $G$  and  $D$ . AnoGAN’s inference cost per sample is therefore  $O(K \cdot (C_G + C_D))$ , where  $C_G, C_D$  are the forward-pass costs of  $G$  and  $D$  respectively. In contrast, f-AnoGAN requires a single forward pass through  $E$ ,  $G$ , and  $D$ , yielding inference cost  $O(C_E + C_G + C_D)$ . Since typical values of  $K$  are 200–500, f-AnoGAN achieves a speedup of two to three orders of magnitude.

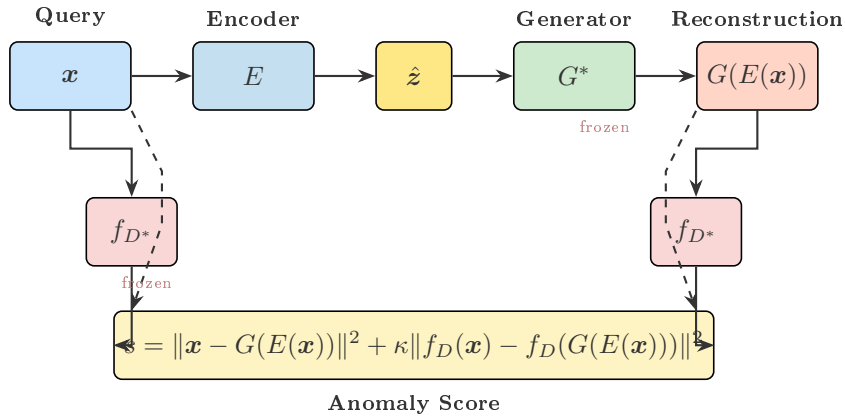


Figure 2: f-AnoGAN inference pipeline. The encoder  $E$  maps the query  $x$  to latent code  $\hat{z}$ ; the frozen generator  $G^*$  reconstructs. The anomaly score combines pixel-wise and discriminator feature-space distances—all in a single forward pass.

## 4 Wasserstein GAN with Gradient Penalty (WGAN-GP)

In practice, training GANs with the original objective (Eq. 1) is notoriously unstable, suffering from mode collapse and vanishing gradients for the generator. The Wasserstein GAN framework addresses these issues.

### Definition 4.1: Wasserstein Distance (Earth-Mover Distance)

The **Wasserstein-1 distance** (or Earth-Mover distance) between two probability distributions  $P$  and  $Q$  on  $\mathbb{R}^d$  is:

$$W_1(P, Q) = \inf_{\gamma \in \Pi(P, Q)} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} [\|\mathbf{x} - \mathbf{y}\|], \quad (7)$$

where  $\Pi(P, Q)$  is the set of all joint distributions (couplings) with marginals  $P$  and  $Q$ .

### Theorem 4.1: Kantorovich–Rubinstein Duality

The Wasserstein-1 distance admits the dual representation:

$$W_1(P, Q) = \sup_{\|C\|_L \leq 1} \mathbb{E}_{\mathbf{x} \sim P}[C(\mathbf{x})] - \mathbb{E}_{\mathbf{y} \sim Q}[C(\mathbf{y})], \quad (8)$$

where the supremum is over all 1-Lipschitz functions  $C : \mathbb{R}^d \rightarrow \mathbb{R}$ .

This motivates replacing the discriminator (which outputs a probability) with a **critic**  $C$  (which outputs an unbounded real-valued score) and enforcing the Lipschitz constraint.

### Definition 4.2: WGAN-GP

The **Wasserstein GAN with Gradient Penalty** [4] trains a critic  $C$  and generator  $G$  with the following objectives:

$$\mathcal{L}_C = \mathbb{E}_{\tilde{\mathbf{x}} \sim p_G}[C(\tilde{\mathbf{x}})] - \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[C(\mathbf{x})] + \lambda_{\text{GP}} \underbrace{\mathbb{E}_{\hat{\mathbf{x}}}[(\|\nabla_{\hat{\mathbf{x}}} C(\hat{\mathbf{x}})\|_2 - 1)^2]}_{\text{gradient penalty}}, \quad (9)$$

$$\mathcal{L}_G = -\mathbb{E}_{\mathbf{z} \sim p_z}[C(G(\mathbf{z}))], \quad (10)$$

where  $\hat{\mathbf{x}} = \alpha \mathbf{x} + (1 - \alpha)\tilde{\mathbf{x}}$  with  $\alpha \sim \text{Uniform}(0, 1)$  is a random interpolation between real and generated samples. The gradient penalty softly enforces the 1-Lipschitz constraint on  $C$ .

### Remark 4.1: Advantages of WGAN-GP over vanilla GAN

The WGAN-GP framework offers several practical advantages:

- The Wasserstein distance provides a meaningful and continuous training signal even when  $p_G$  and  $p_{\text{data}}$  have non-overlapping supports (where the Jensen–Shannon divergence saturates).
- The gradient penalty avoids the instabilities of weight clipping (used in the original WGAN).
- Training is more stable and less sensitive to architecture and hyperparameter choices, with reduced mode collapse.

### Theorem 4.2: Wasserstein Distance: Metrisation of Weak Convergence

Let  $\{P_n\}$  be a sequence of probability distributions on  $\mathbb{R}^d$  with finite first moments. Then:

$$W_1(P_n, P) \rightarrow 0 \iff P_n \xrightarrow{d} P \text{ and } \mathbb{E}_{P_n}[\|\mathbf{x}\|] \rightarrow \mathbb{E}_P[\|\mathbf{x}\|].$$

That is,  $W_1$  metrises weak convergence plus convergence of first moments. This is why the

Wasserstein loss provides a meaningful training signal throughout learning: as the generator improves,  $W_1(p_G, p_{\text{data}})$  decreases continuously.

#### Proposition 4.1: JSD Failure for Low-Dimensional Supports

If  $p_{\text{data}}$  is supported on a manifold  $\mathcal{M}$  of dimension  $k < d$  and  $p_G$  is supported on a manifold  $\mathcal{M}'$  of dimension  $k' < d$ , then generically  $\mathcal{M} \cap \mathcal{M}' = \emptyset$  (they have zero measure of overlap in  $\mathbb{R}^d$ ). In this case:

$$\text{JSD}(p_{\text{data}} \| p_G) = \log 2 \quad (\text{constant, regardless of how close the manifolds are}),$$

and the optimal discriminator achieves perfect classification with probability 1. The generator receives zero useful gradient signal. In contrast,  $W_1(p_{\text{data}}, p_G)$  remains well-defined and varies continuously with the distance between the manifolds, providing informative gradients even in this regime.

#### Remark 4.2: Practical implication for anomaly detection

In anomaly detection, the data often lie on a low-dimensional manifold (normal patterns form a structured subspace of  $\mathbb{R}^{d_x}$ ). The WGAN-GP is particularly well-suited to this setting because its loss function can guide the generator to cover this manifold even when it is thin, whereas a vanilla GAN might suffer from the JSD saturation described above.

## Part II

# Graph Anomaly Detection

## 5 Graphs: Notation and Terminology

#### Definition 5.1: Attributed Graph

An **attributed graph** is a tuple  $\mathcal{G} = (V, E, \mathbf{X})$  where:

- $V = \{v_1, \dots, v_n\}$  is a set of  $n$  nodes.
- $E \subseteq V \times V$  is a set of edges. The **adjacency matrix**  $\mathbf{A} \in \{0, 1\}^{n \times n}$  has  $A_{ij} = 1$  if  $(v_i, v_j) \in E$ .
- $\mathbf{X} \in \mathbb{R}^{n \times d}$  is the **node feature matrix**, where  $\mathbf{x}_i \in \mathbb{R}^d$  is the attribute vector of node  $v_i$ .

The **degree matrix**  $\mathbf{D} = \text{diag}(d_1, \dots, d_n)$  has  $d_i = \sum_j A_{ij}$ .

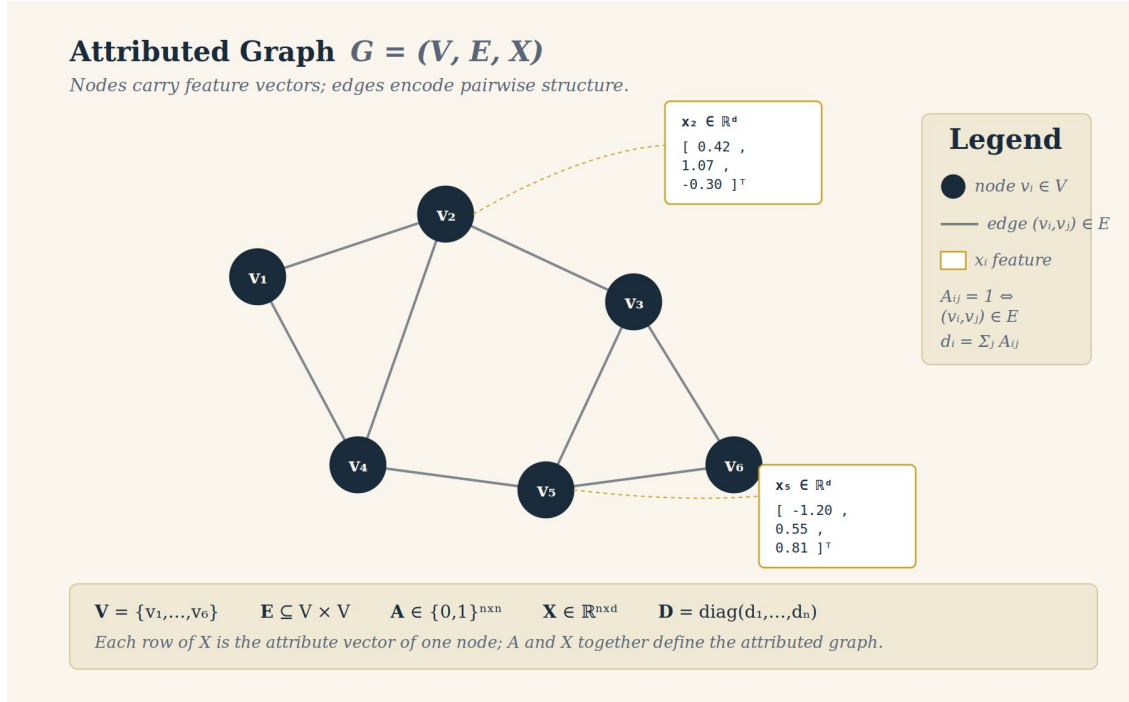


Figure 3: An attributed graph on six nodes. The graph topology is encoded by the adjacency matrix  $A \in \{0,1\}^{n \times n}$  (here  $n = 6$ ); each row of the feature matrix  $X \in \mathbb{R}^{n \times d}$  holds the attribute vector  $x_i$  of one node (two such vectors are pulled out for  $v_2$  and  $v_5$ ). Throughout the chapter, anomaly detectors will exploit either the topological signal carried by  $A$  (structure-based methods), the feature signal carried by  $X$  (attribute-based scoring), or both jointly (deep methods such as DOMINANT).

## 6 Types of Graph Anomalies

Graph anomalies are best classified by the structural *level* at which they manifest — a single node, a single edge, an induced subgraph, or an entire graph viewed as one point in a population of graphs. The level dictates both the modelling assumption (*what counts as normal?*) and the detection technique (*which graph object do we score?*), so it is the most useful first cut through the literature. We give a formal definition for each of the four levels, accompanied by a prototypical example.

### 6.1 Node-Level Anomalies

#### Definition 6.1: Node Anomaly

A **node anomaly** is a vertex  $v_i \in V$  whose local context — topology, attributes, or both — deviates significantly from a reference distribution learned over the remaining vertices. Formally, given a scoring function  $s : V \rightarrow \mathbb{R}_{\geq 0}$  computed from the ego-net or feature neighbourhood of each node,  $v_i$  is anomalous when  $s(v_i)$  exceeds a threshold or lies in a high quantile of  $\{s(v_j)\}_{v_j \in V}$ . Typical signals include:

- a degree  $k(v_i) = \sum_j A_{ij}$  that is much larger or smaller than the local mean,
- a clustering coefficient  $C(v_i)$  that is very low for a high-degree node,
- an attribute vector  $x_i$  that is far from the local average  $\frac{1}{|\mathcal{N}(v_i)|} \sum_{v_j \in \mathcal{N}(v_i)} x_j$ .

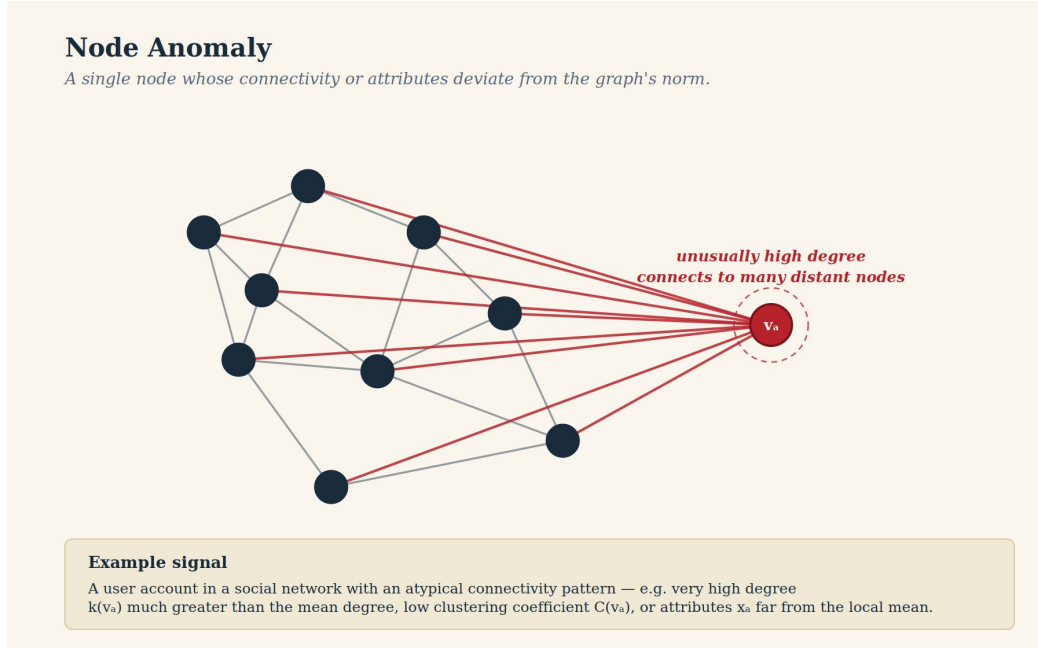


Figure 4: Node anomaly. The red vertex  $v_a$  has an unusually high degree and connects to many otherwise distant nodes, violating the local homogeneity of the surrounding graph. In a social network this could be a sockpuppet or spam account; in a citation network it could be a paper with implausibly many out-of-field citations. Detection signals are precisely those listed in Definition 6.1: degree, clustering coefficient, attribute deviation.

## 6.2 Edge-Level Anomalies

### Definition 6.2: Edge Anomaly

An **edge anomaly** is an edge  $e = (v_i, v_j) \in E$  that is unlikely under a generative model  $\mathbb{P}(\cdot \mid \mathbf{A}, \mathbf{X})$  of the surrounding graph. Equivalently,  $e$  scores poorly under a link-prediction model trained on the rest of the graph: given the embeddings  $\mathbf{z}_i, \mathbf{z}_j$  produced by such a model, the anomaly score is, e.g.,  $s(e) = -\log \mathbb{P}(A_{ij} = 1 \mid \mathbf{z}_i, \mathbf{z}_j)$ . Edge anomalies often bridge regions that the global structure (a stochastic block model, a community decomposition) would predict to remain disjoint.

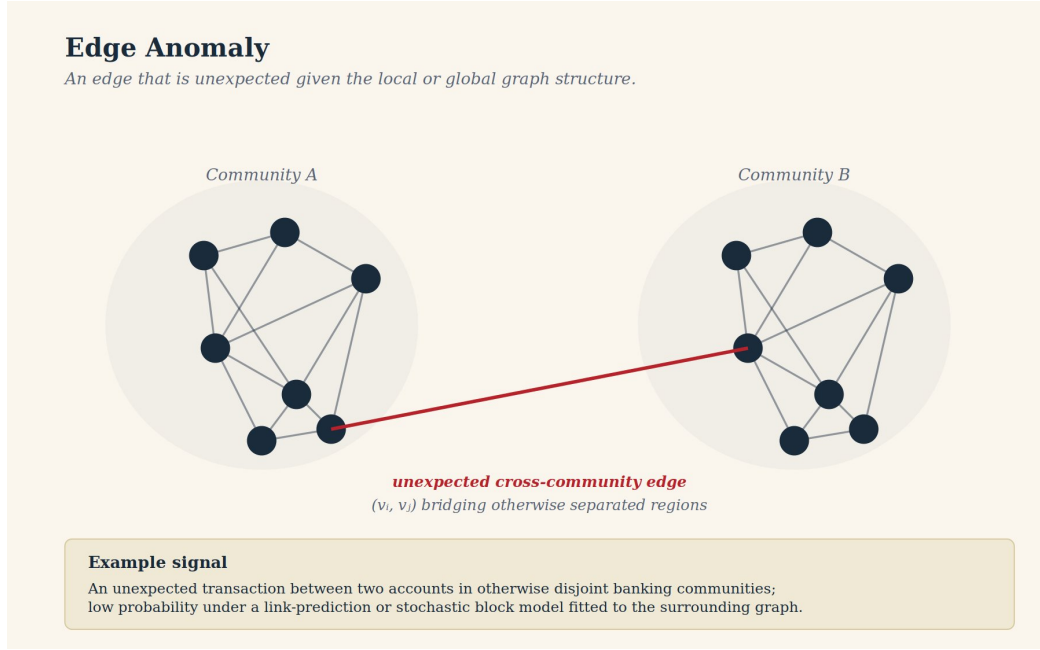


Figure 5: Edge anomaly. An unexpected edge bridges two otherwise well-separated communities. Under a stochastic block model fitted on the two communities the cross-community link has very low probability, and a link-prediction model trained on the rest of the graph assigns it a high reconstruction error. Typical application: flagging a single transaction between two clients who belong to disjoint customer segments.

### 6.3 Subgraph-Level Anomalies

#### Definition 6.3: Subgraph Anomaly

A **subgraph anomaly** is a subset of vertices  $S \subseteq V$  such that the induced subgraph  $\mathcal{G}[S] = (S, E \cap (S \times S))$  exhibits structural properties that are atypical with respect to a null model of the host graph. Common indicators are:

- induced density  $\rho(S) = \frac{|E \cap (S \times S)|}{\binom{|S|}{2}}$  close to 1 (a near-clique) while the global density is much smaller,
- edge count  $e_S$  and node count  $n_S$  that violate the empirical ODDBALL power-law  $e_S \propto n_S^{1.5}$  on ego-nets,
- internal cut value far from what spectral or modularity-based community detection expects.

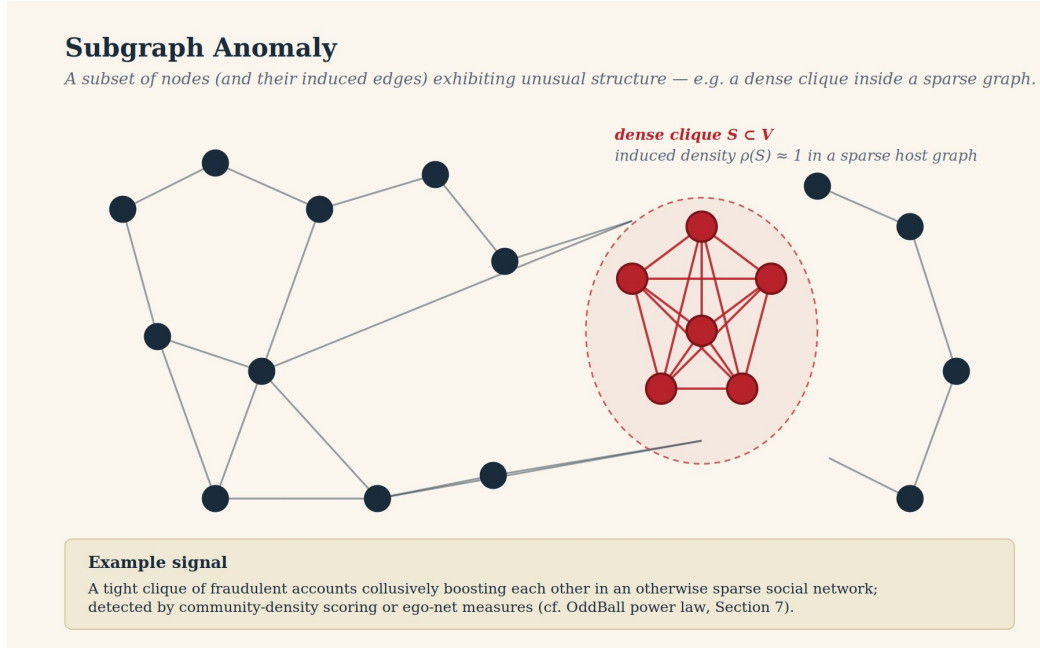


Figure 6: Subgraph anomaly. A dense clique  $S \subset V$  embedded in a sparse host graph attains induced density  $\rho(S) \approx 1$ , well above what one would expect from the rest of the topology. Such patterns frequently arise from collusion — e.g. farms of fake reviewer accounts or coordinated retweet rings. Detection uses ego-net statistics, dense subgraph mining, or community-density scoring.

## 6.4 Whole-Graph-Level Anomalies

### Definition 6.4: Whole-Graph Anomaly

Given a population of graphs  $\{\mathcal{G}_1, \dots, \mathcal{G}_N\}$  drawn from a reference distribution and a graph-level descriptor  $\Phi : \mathcal{G} \mapsto \mathbb{R}^p$  (e.g. a spectral signature, a graphlet count vector, a Weisfeiler–Leman embedding, or a learned pooling output of a GNN), a **whole-graph anomaly** is a graph  $\mathcal{G}^*$  whose descriptor  $\Phi(\mathcal{G}^*)$  lies in a low-density region of the empirical distribution of  $\{\Phi(\mathcal{G}_i)\}_{i=1}^N$ . Anomaly is therefore defined at the level of the entire object, not of individual nodes or edges.

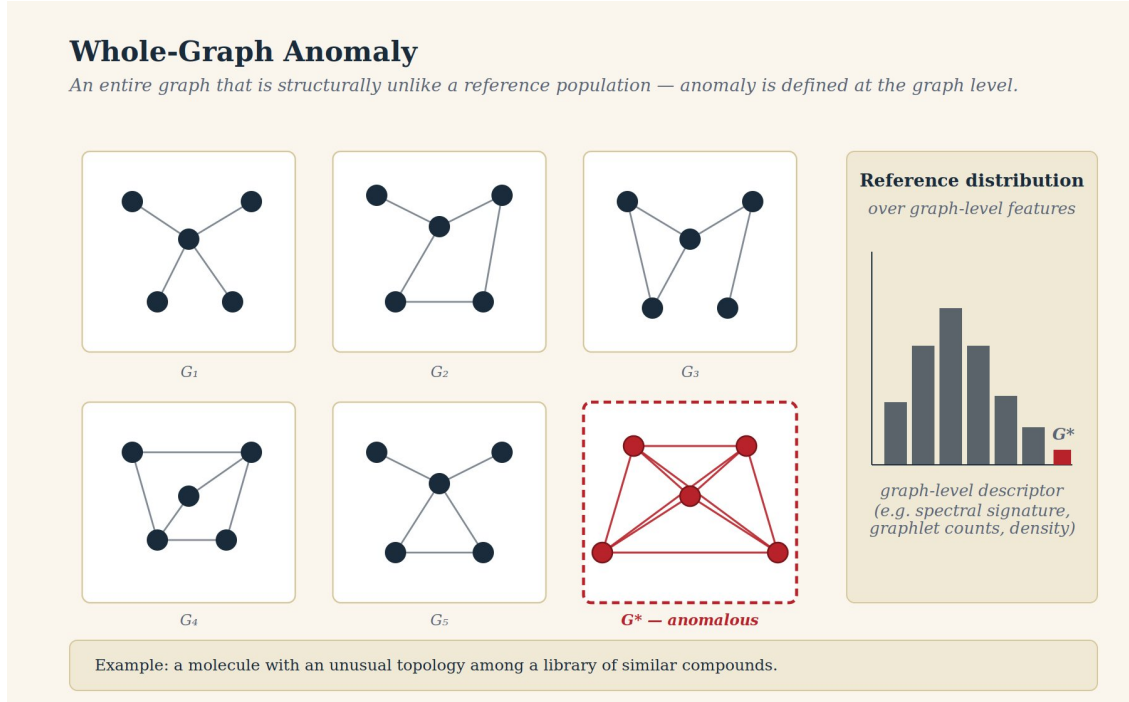


Figure 7: Whole-graph anomaly. Among a reference population  $\{\mathcal{G}_1, \dots, \mathcal{G}_5\}$  of small graphs, the graph  $\mathcal{G}^*$  is structurally atypical: its descriptor (e.g. a spectral signature or a graphlet-count vector) falls in the right tail of the empirical distribution shown on the right. A canonical application is identifying a molecule with an unusual scaffold inside a library of related compounds.

#### Remark 6.1: Detection regime depends on the level

The four levels above each call for a different supervision regime and a different scoring object. Node and edge anomalies are detected *within* a single graph, by comparing each candidate to its neighbourhood; subgraph anomalies require searching over  $V$  for atypical subsets and are combinatorial in nature; whole-graph anomalies are the only ones for which the natural i.i.d. assumption of classical unsupervised anomaly detection holds, because the unit of analysis is the entire graph. The methods covered in the rest of the chapter — feature-based scoring, spectral methods, and DOMINANT — target the node and edge levels.

## 7 Feature-Based Graph Anomaly Detection

The simplest approach to node-level anomaly detection on graphs is to extract structural features for each node and then apply a standard anomaly detection algorithm (e.g. Isolation Forest) to the resulting feature vectors.

### 7.1 Structural Node Features

#### Definition 7.1: Degree

The **degree** of node  $v_i$  is the number of its neighbours:

$$k_i = \sum_{j=1}^n A_{ij} = |\{v_j \in V : (v_i, v_j) \in E\}|. \quad (11)$$

**Definition 7.2: Clustering Coefficient**

The **local clustering coefficient** of node  $v_i$  measures the fraction of its neighbour pairs that are connected:

$$C_i = \frac{|\{(v_j, v_k) \in E : v_j, v_k \in \mathcal{N}(i)\}|}{\binom{k_i}{2}}, \quad (12)$$

where  $\mathcal{N}(i) = \{v_j : A_{ij} = 1\}$  is the set of neighbours of  $v_i$ , and  $\binom{k_i}{2} = \frac{k_i(k_i-1)}{2}$  is the maximum number of edges among them.  $C_i \in [0, 1]$ , with  $C_i = 1$  indicating that  $v_i$ 's neighbourhood forms a complete subgraph (clique).

**Definition 7.3: Average Neighbour Degree**

The **average neighbour degree** of node  $v_i$  is:

$$\bar{k}_{\text{nn},i} = \frac{1}{k_i} \sum_{v_j \in \mathcal{N}(i)} k_j. \quad (13)$$

It measures whether a node's neighbours are themselves well-connected (assortative mixing) or poorly connected (disassortative mixing).

**Definition 7.4: Ego-Network and Ego-Net Density**

The **ego-network** (or ego-net) of node  $v_i$  is the subgraph induced by  $v_i$  and all its neighbours:  $\mathcal{G}_i = \mathcal{G}[\{v_i\} \cup \mathcal{N}(i)]$ . The **ego-net density** is:

$$\rho_i = \frac{|\{(v_j, v_k) \in E : v_j, v_k \in \{v_i\} \cup \mathcal{N}(i)\}|}{\binom{k_i+1}{2}}. \quad (14)$$

**Proposition 7.1: OddBall Power Law**

In many real-world graphs, the relationship between the number of nodes  $n_i$  and the number of edges  $e_i$  in the ego-network of node  $v_i$  follows a power law [5]:

$$e_i \propto n_i^\alpha, \quad \alpha \approx 1.5. \quad (15)$$

Nodes that significantly deviate from this power law—i.e. those with an unusually high or low ratio  $e_i/n_i^\alpha$ —are flagged as structurally anomalous. In particular, nodes embedded in dense cliques produce ego-nets with  $e_i \gg n_i^{1.5}$  (“near-cliques”), while star-like hub nodes show  $e_i \ll n_i^{1.5}$  (“near-stars”).

**Remark 7.1: Feature-based pipeline**

The feature-based approach proceeds as follows:

1. For each node  $v_i$ , compute a structural feature vector  $\mathbf{f}_i = (k_i, C_i, \bar{k}_{\text{nn},i}, \rho_i, \dots)$ .
2. Apply a standard anomaly detector (e.g. Isolation Forest, One-Class SVM) to the resulting feature matrix  $\mathbf{F} \in \mathbb{R}^{n \times p}$ .
3. Rank nodes by anomaly score.

This is simple and interpretable, but it relies on hand-crafted features and may miss patterns that require combining structural and attribute information in nonlinear ways.

## 8 Spectral Methods for Graph Anomaly Detection

## 8.1 The Graph Laplacian

### Definition 8.1: Normalised Graph Laplacian

The **symmetric normalised graph Laplacian** of a graph  $\mathcal{G}$  with adjacency matrix  $\mathbf{A}$  and degree matrix  $\mathbf{D}$  is:

$$\mathbf{L}_{\text{sym}} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}. \quad (16)$$

It is a positive semi-definite matrix with eigenvalues  $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n \leq 2$ .

### Proposition 8.1: Properties of the Laplacian Spectrum

The eigenvalues and eigenvectors of  $\mathbf{L}_{\text{sym}}$  encode fundamental structural properties of the graph:

1. The multiplicity of the eigenvalue  $\lambda = 0$  equals the number of connected components.
2. The **Fiedler value**  $\lambda_2$  (the smallest nonzero eigenvalue) measures graph connectivity: a small  $\lambda_2$  indicates the graph can be easily partitioned into loosely connected parts.
3. Tight clusters manifest as *gaps* in the eigenspectrum.
4. The eigenvectors associated with the smallest eigenvalues provide a low-dimensional embedding of the nodes that preserves global graph structure (**spectral embedding**).

### Theorem 8.1: Cheeger Inequality

Let  $\mathcal{G}$  be a connected graph and let  $h(\mathcal{G})$  denote its **Cheeger constant** (or isoperimetric number), defined as:

$$h(\mathcal{G}) = \min_{\emptyset \neq S \subset V, |S| \leq n/2} \frac{|\partial S|}{\min(\text{vol}(S), \text{vol}(\bar{S}))},$$

where  $\partial S$  is the set of edges crossing the cut  $(S, \bar{S})$  and  $\text{vol}(S) = \sum_{v_i \in S} d_i$  is the volume of  $S$ . Then the Fiedler value  $\lambda_2$  of  $\mathbf{L}_{\text{sym}}$  satisfies:

$$\frac{h(\mathcal{G})^2}{2} \leq \lambda_2 \leq 2h(\mathcal{G}). \quad (17)$$

This connects the algebraic quantity  $\lambda_2$  to the combinatorial notion of graph expansion. A small  $\lambda_2$  implies the existence of a sparse cut, which may signal the presence of anomalous substructures (e.g. loosely connected communities or bridge nodes).

## 8.2 Spectral Anomaly Detection

### Definition 8.2: Spectral Embedding for Anomaly Detection

Given the  $k$  eigenvectors  $\mathbf{u}_1, \dots, \mathbf{u}_k$  of  $\mathbf{L}_{\text{sym}}$  corresponding to its  $k$  smallest eigenvalues, the **spectral embedding** of node  $v_i$  is the vector:

$$\mathbf{e}_i = (u_{1,i}, u_{2,i}, \dots, u_{k,i}) \in \mathbb{R}^k. \quad (18)$$

Anomalous nodes—such as hubs, bridges, or nodes in anomalous cliques—typically have **unusual spectral coordinates** compared to regular nodes, and can be detected by applying a density-based or reconstruction-based anomaly detector to the embedding space.

### Remark 8.1: Spectral vs feature-based approaches

Spectral methods are *unsupervised* and require no hand-crafted features; they work directly from the adjacency matrix. However, they capture only topological information and ignore

node attributes. For attributed graphs, methods that combine structural and attribute signals (such as GNN-based autoencoders) are generally more powerful.

## 9 Deep Graph Anomaly Detection: DOMINANT

We now introduce deep learning methods for graph anomaly detection, focusing on DOMINANT [7], which uses a Graph Convolutional Network (GCN) autoencoder.

### 9.1 Graph Convolutional Networks (GCN)

The GCN of Kipf and Welling [6] is the workhorse encoder of DOMINANT and of most early deep methods for attributed-graph anomaly detection. Although the layer can be stated in one line, every ingredient (the self-loops, the symmetric normalisation, the choice to use a single matrix multiplication) is the result of a sequence of approximations starting from spectral graph theory. Understanding this derivation is what makes the layer's behaviour — and its failure modes — transparent.

#### 9.1.1 From spectral filters to a localised polynomial

A signal on the graph is a vector  $\mathbf{x} \in \mathbb{R}^n$  assigning a real number to each node. The graph Fourier transform diagonalises  $\mathbf{L}_{\text{sym}} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$  (with  $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ ,  $0 = \lambda_1 \leq \dots \leq \lambda_n \leq 2$ ) and writes  $\mathbf{x}$  in the eigenbasis:  $\hat{\mathbf{x}} = \mathbf{U}^\top \mathbf{x}$ . A **spectral graph convolution** with filter  $g_\theta : [0, 2] \rightarrow \mathbb{R}$  acts component-wise in this basis:

$$g_\theta(\mathbf{L}_{\text{sym}}) \mathbf{x} = \mathbf{U} g_\theta(\mathbf{\Lambda}) \mathbf{U}^\top \mathbf{x}. \quad (19)$$

Using (19) as a learnable layer is impractical: each application requires the full eigendecomposition (cost  $O(n^3)$ ), the filter is dense in the spectral domain (no spatial localisation), and the parameters depend on  $n$ .

The trick used by ChebNet [6] is to restrict  $g_\theta$  to a polynomial of degree  $K$ :

$$g_\theta(\mathbf{L}_{\text{sym}}) = \sum_{k=0}^K \theta_k \mathbf{L}_{\text{sym}}^k. \quad (20)$$

Two structural properties immediately follow:

- **Localisation:** since  $(\mathbf{L}_{\text{sym}}^k)_{ij} = 0$  whenever the graph distance  $d(v_i, v_j) > k$ , the filter  $g_\theta(\mathbf{L}_{\text{sym}})$  depends only on  $K$ -hop neighbourhoods.
- **No eigendecomposition:** the action of (20) on a signal can be computed by  $K$  sparse matrix-vector products, costing  $O(K \cdot |E|)$ .

#### 9.1.2 The renormalisation trick: deriving the GCN layer

GCN takes a single first-order step ( $K = 1$ ) and applies a chain of simplifications. Starting from (20) with  $K = 1$  and renaming  $\theta_0, \theta_1$ :

$$g_\theta(\mathbf{L}_{\text{sym}}) \mathbf{x} = \theta_0 \mathbf{x} + \theta_1 \mathbf{L}_{\text{sym}} \mathbf{x} = \theta_0 \mathbf{x} + \theta_1 (\mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}) \mathbf{x}. \quad (21)$$

(i) **Parameter sharing.** Constraining  $\theta_0 = -\theta_1 =: \theta$  leaves a single scalar parameter per filter and yields

$$g_\theta(\mathbf{L}_{\text{sym}}) \mathbf{x} = \theta (\mathbf{I} + \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}) \mathbf{x}.$$

(ii) **The renormalisation trick.** The matrix  $\mathbf{I} + \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$  has spectrum in  $[0, 2]$ , which causes activations to grow with each layer and makes deep networks unstable. Kipf and Welling replace it by the structurally analogous but better-behaved

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}, \quad \tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}, \quad (22)$$

i.e. self-loops are added *before* normalising. The eigenvalues of  $\hat{\mathbf{A}}$  now live in  $[-1, 1]$ , fixing the numerical instability. Self-loops also guarantee that each node retains its own information when aggregating from neighbours.

(iii) **Vector-valued features and learnable mixing.** For node features  $\mathbf{H}^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$ , the scalar parameter  $\theta$  is replaced by a learnable matrix  $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell+1}}$  that mixes channels. Composing with a non-linearity yields the standard GCN layer.

#### Definition 9.1: GCN Layer

A **Graph Convolutional Network layer** [6] propagates and transforms node representations using the graph structure. Given a node feature matrix  $\mathbf{H}^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$  at layer  $\ell$  and the renormalised adjacency  $\hat{\mathbf{A}}$  defined in (22), the GCN layer computes:

$$\mathbf{H}^{(\ell+1)} = \sigma(\hat{\mathbf{A}} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)}), \quad (23)$$

where  $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell+1}}$  is a learnable weight matrix and  $\sigma$  is a nonlinear activation (e.g. ReLU). By convention  $\mathbf{H}^{(0)} = \mathbf{X}$ .

#### 9.1.3 Node-wise rewriting and message passing

Equation (23) is compact at the matrix level but its behaviour is clearer node by node. Writing  $\hat{A}_{ij} = (\tilde{d}_i \tilde{d}_j)^{-1/2} \tilde{A}_{ij}$ , the update for node  $v_i$  is:

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left( \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}} \mathbf{W}^{(\ell)\top} \mathbf{h}_j^{(\ell)} \right). \quad (24)$$

#### Remark 9.1: Message-passing interpretation

The matrix product  $\hat{\mathbf{A}} \mathbf{H}^{(\ell)}$  computes, for each node  $v_i$ , a normalised weighted average of its own features and those of its neighbours — equation (24). This is a first-order *message passing* operation: each node aggregates information from its 1-hop neighbourhood. Stacking  $L$  GCN layers expands the receptive field to  $L$ -hop neighbourhoods. Three structural choices baked into (24) are worth highlighting:

- **Self-loop:** the sum runs over  $\mathcal{N}(i) \cup \{i\}$ , so  $v_i$  always receives its own message.
- **Symmetric normalisation:** the coefficient  $1/\sqrt{\tilde{d}_i \tilde{d}_j}$  down-weights edges incident to high-degree (hub) nodes both on the sending and receiving end. Compared to the row-stochastic alternative  $1/\tilde{d}_i$  (random-walk normalisation), it preserves the symmetry of the operator and prevents the hidden representations of hubs from dominating.
- **Same weight matrix for self and neighbours:** unlike GraphSAGE or GAT, GCN uses a single  $\mathbf{W}^{(\ell)}$  for every term in the sum — this saves parameters but limits expressive power, see Section 9.1.5.

### 9.1.4 Receptive field, computational cost, and the role of self-loops

#### Proposition 9.1: Receptive field of an $L$ -layer GCN

After  $L$  GCN layers, the representation  $\mathbf{h}_i^{(L)}$  depends on the input features of every node  $v_j$  within graph distance  $L$  of  $v_i$ :

$$\mathbf{h}_i^{(L)} = F(\{\mathbf{x}_j : d(v_i, v_j) \leq L\}; \{\mathbf{W}^{(\ell)}\}_{\ell=0}^{L-1}).$$

This follows directly from  $(\hat{\mathbf{A}}^L)_{ij} = 0$  when  $d(v_i, v_j) > L$ .

**Cost.** Each layer is dominated by two operations. The neighbourhood aggregation  $\hat{\mathbf{A}}\mathbf{H}^{(\ell)}$  exploits the sparsity of  $\hat{\mathbf{A}}$  (the same as  $\mathbf{A}$  plus the diagonal) and costs  $O(|E| \cdot d_\ell)$ , while the linear transformation  $\mathbf{W}^{(\ell)}$  costs  $O(n \cdot d_\ell d_{\ell+1})$ . An  $L$ -layer GCN therefore costs

$$O(L \cdot |E| \cdot d_{\max}) + O(L \cdot n \cdot d_{\max}^2)$$

per forward pass, where  $d_{\max} = \max_\ell d_\ell$ . This linear scaling in  $|E|$  is the reason GCN can be applied to graphs with millions of edges.

**Why self-loops?** Without the addition  $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ , the update would replace each node feature by a weighted average of its neighbours *only*, discarding the node's own information. Self-loops also restore  $\hat{\mathbf{A}}$  to a strictly positive diagonal, which (together with the renormalisation in (22)) brings the spectrum of  $\hat{\mathbf{A}}$  into  $[-1, 1]$  and stabilises training.

### 9.1.5 Failure modes and limitations

#### Proposition 9.2: Over-Smoothing in Deep GCNs

As the number of GCN layers  $L$  increases, all node representations converge to the same vector. Formally, for a connected non-bipartite graph, as  $L \rightarrow \infty$ :

$$\mathbf{H}^{(L)} \rightarrow \mathbf{1} \mathbf{v}^\top,$$

where  $\mathbf{v}$  is a vector that depends on the graph structure but not on the input features. This phenomenon is called **over-smoothing**: after too many message-passing rounds, all nodes become indistinguishable, destroying the discriminative signal needed for anomaly detection. The reason is spectral: the dominant eigenvalue of  $\hat{\mathbf{A}}$  is 1 with eigenvector proportional to  $\tilde{\mathbf{D}}^{1/2} \mathbf{1}$ , and all other eigenvalues lie strictly inside  $(-1, 1)$ , so iterating  $\hat{\mathbf{A}}$  projects onto the leading eigenspace. In practice, GCN-based anomaly detectors (including DOMINANT) use only 2–3 layers to balance expressiveness against over-smoothing.

#### Remark 9.2: Connection to random walks

The propagation rule with the row-stochastic normalisation  $\mathbf{D}^{-1} \mathbf{A}$  is one step of a simple random walk; with self-loops it becomes a *lazy* walk. Over-smoothing is then the neural-network analogue of mixing to the stationary distribution: after enough steps, the walk “forgets” its starting point. Symmetric normalisation  $\hat{\mathbf{A}}$  yields a closely related, similar matrix sharing the same spectrum, hence the same mixing behaviour. This explains why *shallow* GCNs (few steps = local information) are preferable for anomaly detection, where deviations are local by nature.

**Remark 9.3: Other limitations of GCN**

Beyond over-smoothing, the GCN layer of (23) has structural shortcomings that motivate later architectures:

- **Fixed aggregation weights.** The coefficient  $1/\sqrt{\tilde{d}_i\tilde{d}_j}$  depends only on the graph topology, not on the features. GAT replaces it by a learned attention coefficient  $\alpha_{ij}(\mathbf{h}_i, \mathbf{h}_j)$ .
- **Same transformation for self and neighbours.** GCN applies the same  $\mathbf{W}^{(\ell)}$  to  $\mathbf{h}_i^{(\ell)}$  and to its neighbours. GraphSAGE separates the two (CONCAT of self and aggregated message), increasing expressiveness.
- **No edge features.** Equation (23) only sees the binary (or unweighted) presence of edges. MPNN and EGNN-style models pass edge attributes through a learnable function  $\psi(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij})$ .
- **Originally transductive.** Training on the full graph hard-codes the topology; inductive variants (GraphSAGE, FastGCN) sample neighbourhoods to generalise to unseen nodes.
- **Bounded expressiveness.** The discriminative power of GCN (and of any sum/mean message-passing scheme that normalises by degree) is upper-bounded by the 1-Weisfeiler–Leman graph isomorphism test: there exist non-isomorphic graphs that no GCN can distinguish. GIN closes this gap by using sums and an MLP.

For graph anomaly detection, the most directly relevant of these limitations are the fixed weights (which prevent the model from emphasising suspicious edges) and over-smoothing (which forces shallow architectures).

**9.2 DOMINANT Architecture****Definition 9.2: DOMINANT**

**DOMINANT** (Deep Anomaly Detection on Attributed Networks) [7] is a GCN-based autoencoder that jointly reconstructs both node attributes and graph structure. It consists of:

1. A **GCN encoder** that produces node embeddings:

$$\mathbf{Z} = \text{GCN}_{\text{enc}}(\mathbf{X}, \hat{\mathbf{A}}) \in \mathbb{R}^{n \times d_z}. \quad (25)$$

2. An **attribute decoder** that reconstructs node features:

$$\hat{\mathbf{X}} = \sigma(\mathbf{Z} \mathbf{W}_{\text{attr}}). \quad (26)$$

3. A **structure decoder** that reconstructs the adjacency matrix via inner products:

$$\hat{\mathbf{A}} = \sigma(\mathbf{Z} \mathbf{Z}^\top). \quad (27)$$

**Definition 9.3: DOMINANT Anomaly Score**

The anomaly score for node  $v_i$  is a convex combination of attribute and structural reconstruction errors:

$$s(v_i) = \alpha \underbrace{\|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2}_{\text{attribute score}} + (1 - \alpha) \underbrace{\|\mathbf{a}_i - \hat{\mathbf{a}}_i\|^2}_{\text{structure score}}, \quad (28)$$

where  $\mathbf{a}_i$  and  $\hat{\mathbf{a}}_i$  are the  $i$ -th rows of  $\mathbf{A}$  and  $\hat{\mathbf{A}}$  respectively, and  $\alpha \in [0, 1]$  is a hyperparameter that controls the relative importance of attribute vs structural anomalies.

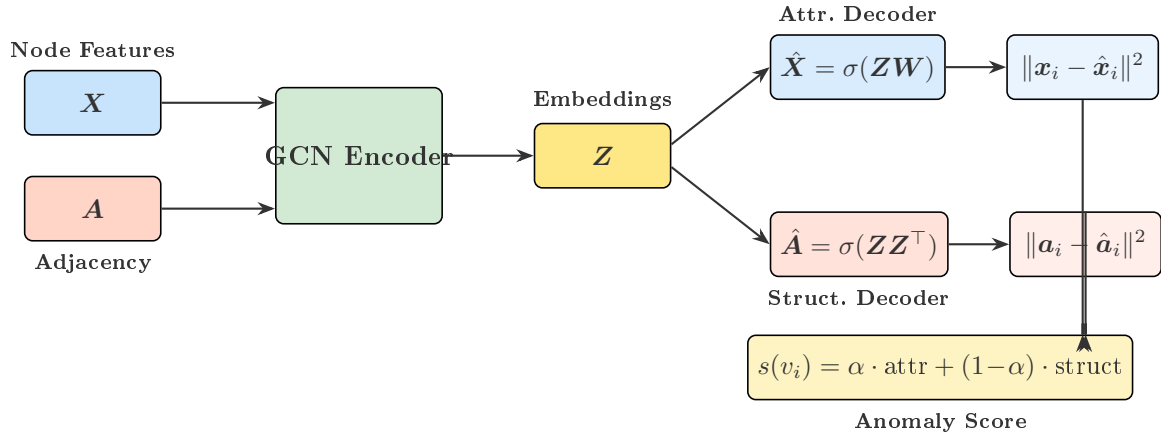


Figure 8: DOMINANT architecture. A GCN encoder produces node embeddings  $Z$ . Two decoders reconstruct node attributes ( $\hat{X}$ ) and graph structure ( $\hat{A}$ ) respectively. The anomaly score is a weighted combination of both reconstruction errors.

#### Remark 9.4: Role of $\alpha$ in DOMINANT

The parameter  $\alpha$  should be tuned based on the nature of the anomalies:

- $\alpha \rightarrow 1$ : emphasises attribute anomalies (nodes with unusual features).
- $\alpha \rightarrow 0$ : emphasises structural anomalies (nodes with unusual connectivity).
- $\alpha = 0.5$ : balanced weighting; a reasonable default in the absence of domain knowledge.

When ground-truth labels are available for a validation set,  $\alpha$  can be tuned to maximise AUC-ROC.

## 10 Other Deep Graph Anomaly Detection Methods

Beyond DOMINANT, several other deep methods have been proposed. We briefly survey the main families.

### 10.1 Contrastive Methods

#### Definition 10.1: Contrastive Graph Anomaly Detection

**Contrastive** methods detect anomalies by learning representations such that normal nodes are similar to their local context (neighbourhood subgraph) while anomalous nodes are dissimilar. The key idea is to maximise agreement between a node's representation and its context representation (positive pairs) while minimising agreement with random contexts (negative pairs).

Notable contrastive methods include:

- **CoLA** [8]: contrasts a node with its context subgraph via a GNN encoder and a bilinear discriminator.
- **ANEMONE** [9]: extends contrastive learning to multiple scales (node-level and subgraph-level), providing richer anomaly signals.
- **GRADATE** [10]: uses a hierarchical architecture with dual discriminators for multi-scale contrastive anomaly detection.

### 10.2 Dual Autoencoder Methods

**AnomalyDAE** [11] uses separate autoencoders for structure and attributes, with a cross-attention mechanism to capture dependencies between the two modalities. Each autoencoder produces its

own anomaly score, and the final score is a combination.

## 11 Method Selection Guide

Table 2 provides practical guidance on selecting an anomaly detection method based on the data type and scenario.

Table 2: Recommended anomaly detection methods by data type and scenario.

| Scenario                                  | Recommended method                      |
|-------------------------------------------|-----------------------------------------|
| Tabular, no labels                        | Isolation Forest                        |
| Tabular, need uncertainty                 | VAE                                     |
| Tabular, need generative model            | f-AnoGAN                                |
| Time series, point anomalies              | LSTM-AE                                 |
| Time series, long dependencies            | Transformer-AE                          |
| Graph, node anomalies, features available | Isolation Forest on structural features |
| Graph, attributed nodes, no labels        | DOMINANT                                |
| Graph, contrastive learning               | CoLA / ANEMONE                          |

## References

## References

- [1] I. Goodfellow *et al.*, “Generative adversarial nets,” *NeurIPS*, 2014.
- [2] T. Schlegl *et al.*, “Unsupervised anomaly detection with generative adversarial networks to guide marker discovery,” *IPMI*, 2017.
- [3] T. Schlegl *et al.*, “f-AnoGAN: Fast unsupervised anomaly detection with generative adversarial networks,” *Medical Image Analysis*, 54:30–44, 2019.
- [4] I. Gulrajani *et al.*, “Improved training of Wasserstein GANs,” *NeurIPS*, 2017.
- [5] L. Akoglu, M. McGlohon, and C. Faloutsos, “OddBall: Spotting anomalies in weighted graphs,” *PAKDD*, 2010.
- [6] T.N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *ICLR*, 2017.
- [7] K. Ding *et al.*, “Deep anomaly detection on attributed networks,” *SDM*, 2019.
- [8] Y. Liu *et al.*, “Anomaly detection on attributed networks via contrastive self-supervised learning,” *IEEE TNNLS*, 2021.
- [9] M. Jin *et al.*, “ANEMONE: Graph anomaly detection with multi-scale contrastive learning,” *CIKM*, 2021.
- [10] J. Duan *et al.*, “GRADATE: Graph anomaly detection via multi-scale contrastive learning with learnable data augmentation,” *ACM TIST*, 2023.
- [11] H. Fan *et al.*, “AnomalyDAE: Dual autoencoder for anomaly detection on attributed networks,” *ICASSP*, 2020.